Reviewer Pack — Minimal Rank of Quantum Cluster States over DPLL Proof Trees

Entry 46a899e5eb6b · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-27 04:04:54 UTC

Minimal Rank of Quantum Cluster States over DPLL Proof Trees

Entry ID: `46a899e5eb6b` **Verdict**: `INCONCLUSIVE` **Recorded**: 2026-05-27 04:04:54 UTC

1. Conjecture

Field A (mathematical branch): Quantum Information Theory (Cluster States) **Field B** (complexity object): Complexity Theory: DPLL Proof Complexity

Statement:

> ['For a given satisfiability instance, the minimal rank of the quantum cluster state corresponding to its solution is upper-bounded by the depth of the DPLL proof tree for that instance.', 'Equivalently, if an instance has a DPLL proof tree with depth D, then there exists a quantum cluster state with rank ≤ 2^D.', 'Furthermore, the minimal rank of such a quantum cluster state provides a lower bound on the size of the smallest possible resolution proof for the same instance.']

Rationale (proposer's reasoning):

> ['Quantum cluster states are highly entangled states that have been shown to be useful in various quantum algorithms. Their minimal rank could potentially provide insights into the complexity of finding solutions to satisfiability problems.', 'The DPLL proof tree captures the structure of a proof for a satisfiability problem and its depth is a measure of the complexity of the problem. If there is a direct relationship between these two quantities, it would suggest that quantum algorithms might be able to solve NP-complete problems efficiently.', 'This conjecture bridges quantum information theory with computational complexity by relating a quantum resource (cluster states) to a classical complexity measure (DPLL proof tree depth).']

Taxonomy category: `Quantum_Information_Theoretic_Bridge` (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): `c5846c7b34fa7a35`

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if the minimal rank of quantum cluster states matches the depth of DPLL proof trees with at least 80% accuracy for all instances, and there are no instances where the minimal rank exceeds 2^(depth of DPLL proof tree) by more than 3.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM₁	LLM₂
RELATIVIZATION	UNCERTAIN	1.00	HITS	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	1.00	SAFE	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: NOVEL against 0 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "quantum cluster states" AND "DPLL proof complexity" - "minimal rank" AND "quantum information theory" AND "DPLL proof tree" - "resolution proof size" AND "cluster state rank" AND "satisfiability instance"

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)
Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution**: rc=1, elapsed=2.0s

5.1 Generated Python source

```python
import random
import math

def gaussian_elimination(A, b):
    n = len(b)
    A_b = [A[i] + [b[i]] for i in range(n)]
    for i in range(n):
        if A_b[i][i] == 0:
            return None  # No unique solution exists
        for j in range(i+1, n):
            factor = A_b[j][i] / A_b[i][i]
            for k in range(i, n + 1):
                A_b[j][k] -= factor * A_b[i][k]
    x = [0] * n
    for i in range(n-1, -1, -1):
        x[i] = (A_b[i][-1] - sum(A_b[i][j] * x[j] for j in range(i+1, n))) / A_b[i][i]
    return x

def matrix_multiplication(A, B):
    m = len(A)
    p = len(B[0])
    q = len(B)
    C = [[0] * p for _ in range(m)]
    for i in range(m):
        for j in range(p):
```

```
            for k in range(q):
                C[i][j] += A[i][k] * B[k][j]
    return C

def dpll(instance, assignment=None):
    if assignment is None:
        assignment = {}
    variables = set()
    for clause in instance:
        variables.update(clause)
    unassigned = variables - set(assignment.keys())
    if not unassigned:
        if all(any(lit in assignment and assignment[lit] == val for lit, val in clause) for clause
            return True
        else:
            return False
    var = next(iter(unassigned))
    for val in [True, False]:
        new_assignment = assignment.copy()
        new_assignment[var] = val
        if dpll(instance, new_assignment):
            return True
    return False

def construct_quantum_cluster_state(instance, assignment):
    n = len(instance)
    m = 2 ** n
    A = [[0] * m for _ in range(m)]
    b = [0] * m
    for i in range(m):
        binary = format(i, f'0{n}b')
        assignment_i = {variables[j]: int(binary[j]) for j in range(n)}
        if dpll(instance, assignment_i):
            A[i][i] = 1
            b[i] = 1
    return A, b

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n = random.randint(5, 40)
    instance = [random.sample(range(-n, n+1), random.randint(2, n)) for _ in range(n)]
    assignment = {i: random.choice([True, False]) for i in range(-n, n+1)}

    A, b = construct_quantum_cluster_state(instance, assignment)
    solution = gaussian_elimination(A, b)
    if solution is None:
        return {
            "metric_name": "minimal_rank",
            "metric_value": float('inf'),
            "instances_tested": 1,
            "conjecture_holds": False,
            "counterexample": "No unique solution exists"
        }

    minimal_rank = len([i for i, val in enumerate(solution) if val != 0])
    depth_of_dpll_tree = dpll(instance)
```

```
    return {
        "metric_name": "minimal_rank",
        "metric_value": minimal_rank,
        "instances_tested": 1,
        "conjecture_holds": minimal_rank <= 2 ** depth_of_dpll_tree and minimal_rank >= depth_of_d
        "counterexample": ""
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [random.getrandbits(32) for _ in range(30)]

    results = []
    total_metric_value = 0
    count_supporting_conjecture = 0

    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)
        total_metric_value += trial_result["metric_value"]
        if trial_result["conjecture_holds"]:
            count_supporting_conjecture += 1

    mean_metric_value = total_metric_value / len(results)
    support_fraction = count_supporting_conjecture / len(results)

    if all(result["conjecture_holds"] for result in results):
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std=0.0 support_fraction={support_fract
    elif support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={mean_metric_value} std=0.0 support_fraction={support_fract
    else:
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conj
        print(f"RESULT: FALSIFIED counterexample=\"minimal_rank exceeds 2^depth_of_dpll_tree by mo
```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```
--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_fa8cd04a.py", line 114, in <module>
    trial_result = run_trial(seed)
                   ^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_fa8cd04a.py", line 83, in run_trial
    A, b = construct_quantum_cluster_state(instance, assignment)
           ^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_fa8cd04a.py", line 67, in construct_qua
    A = [[0] * m for _ in range(m)]
         ~~~~^~~
MemoryError
```

8. Critic adversarial review

Critic verdict: `CHALLENGE`

Critic reasoning:

> skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: `INCONCLUSIVE`

Reasoning:

> The test crashed due to a MemoryError, which means it did not produce any data that could be used to evaluate the conjecture's support or falsification conditions. | next: Investigate the cause of the MemoryError and attempt the test again with increased memory allocation or optimized code.

11. Audit log (LLM calls)

Total LLM calls: 12

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	?	?	0	0	15160
2	propose	ollama_remote	glm4:latest	0	0	13179
3	propose	?	?	0	0	15325
4	propose	ollama_remote	glm4:latest	0	0	14238
5	preregistration	ollama_remote	glm4:latest	0	0	5586
6	novelty	ollama_remote	glm4:latest	0	0	4689
7	novelty	ollama_remote	glm4:latest	0	0	15122
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	13593
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	7952
10	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	12285
11	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	31265
12	judge	ollama_remote	glm4:latest	0	0	53664

Totals: 0 input tokens, 0 output tokens, ~$0.0000 cost, 202060 ms total latency. Provider mix: {'?': 2, 'ollama_remote': 10}

(full prompt+response transcripts available in `research/audit/46a899e5eb6b.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/46a899e5eb6b.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)
Replay tarball: `research/replay/46a899e5eb6b.tar.gz` (if generated)